🤖

# 100+ ML Interview Questions — Complete Prep Guide

> 🚨 **These are not definition questions.** These are the questions interviewers at Google, Meta, Amazon, and every serious data company actually ask. Knowing the concept is not enough — you need to be able to explain it clearly, give an example, and connect it to real decisions.

---

## 🗺️ What's Covered

| Section | Questions | What gets tested |
| --- | --- | --- |
| 🟢 ML Fundamentals | Q1 – Q20 | Core concepts every ML practitioner must know |
| 🔵 Supervised Learning | Q21 – Q40 | Regression, classification, algorithms, evaluation |
| 🟠 Unsupervised Learning | Q41 – Q55 | Clustering, dimensionality reduction, anomaly detection |
| 🟣 Model Evaluation | Q56 – Q70 | Metrics, validation, bias-variance, business alignment |
| 🔴 Feature Engineering | Q71 – Q82 | The skill that actually separates good models from bad ones |
| ⚫ Deep Learning & NLP | Q83 – Q95 | Neural networks, transformers, practical DL questions |
| 🟡 ML System Design | Q96 – Q110 | End-to-end thinking — the senior-level differentiator |

---

## 🟢 ML Fundamentals — Q1 to Q20

> 💡 Get these wrong and the interview ends early. Get these right and you've cleared the first filter most candidates fail.

---

### Q1. What is the difference between supervised, unsupervised, and reinforcement learning?

| Type | Has labels? | Goal | Examples |
|---|---|---|---|
| Supervised | ✅ Yes | Predict a label from inputs | Classification, regression |
| Unsupervised | ❌ No | Find structure in data | Clustering, dimensionality reduction |
| Reinforcement | Environment reward | Maximise cumulative reward | Game playing, robotics, recommendation |

> **In an interview:** Always give a real-world example after defining each one. "Supervised is predicting whether a customer will churn, given their behaviour history."

---

### Q2. What is the bias-variance tradeoff?

- **Bias** — error from wrong assumptions. High bias = model too simple = underfitting.
- **Variance** — error from sensitivity to training data. High variance = model too complex = overfitting.
- **The tradeoff:** Reducing bias tends to increase variance and vice versa. The goal is minimising total error.

| | High Bias | High Variance |
|---|---|---|
| Training error | High | Low |
| Validation error | High | Much higher than training |
| Problem | Underfitting | Overfitting |
| Fix | More complex model, more features | Regularisation, more data, simpler model |

---

### Q3. What is overfitting and how do you prevent it?

Overfitting is when a model learns the training data too well — including noise — and fails to generalise to new data.

**Prevention methods:**

- **More data** — the single best fix
- **Cross-validation** — ensure performance holds on unseen splits

- **Regularisation** — L1 (Lasso), L2 (Ridge) penalise model complexity
- **Dropout** — randomly drop neurons during training (neural networks)
- **Early stopping** — stop training when validation loss stops improving
- **Simpler model** — fewer parameters, shallower trees
- **Feature selection** — remove irrelevant or noisy features

### Q4. What is the difference between parameters and hyperparameters?

| | Parameters | Hyperparameters |
|---|---|---|
| What | Learned by the model during training | Set by you before training |
| Examples | Weights in neural network, coefficients in regression | Learning rate, tree depth, number of clusters |
| How set | Gradient descent / optimisation | Grid search, random search, Bayesian optimisation |

### Q5. Explain the curse of dimensionality.

As the number of features (dimensions) increases, the volume of the feature space grows exponentially. This means:

- Data becomes increasingly sparse — points are far apart
- Distance metrics lose meaning (everything becomes equidistant)
- Models need exponentially more data to generalise
- Overfitting risk increases dramatically

**Fix:** Dimensionality reduction (PCA, t-SNE), feature selection, regularisation.

### Q6. What is cross-validation and why do we use it?

Cross-validation evaluates how well a model generalises by training and testing on different subsets of the data.

```
K-Fold CV (k=5):
Fold 1: [Test] [Train] [Train] [Train] [Train]
Fold 2: [Train] [Test] [Train] [Train] [Train]
Fold 3: [Train] [Train] [Test] [Train] [Train]
... repeat for all k folds, average the results
```

| Method | When to use |
|---|---|
| K-Fold (k=5 or 10) | Standard default |
| Stratified K-Fold | Class imbalance — preserves class proportions |
| Time Series Split | Time-ordered data — never use future data to predict past |
| Leave-One-Out | Very small datasets |

### Q7. What is the train/validation/test split and why do you need all three?

| Set | Used for | Size (typical) |
|---|---|---|
| Training | Fit model parameters | 60–70% |
| Validation | Tune hyperparameters, select model | 10–20% |
| Test | Final, unbiased evaluation — touch once only | 10–20% |

> **Critical:** If you use the test set during development, it becomes a validation set and your final evaluation is optimistically biased. The test set is sacred.

### Q8. What is gradient descent?

An optimisation algorithm that minimises a loss function by iteratively adjusting parameters in the direction of steepest descent.

```
w = w - learning_rate × ∂Loss/∂w
```

| Variant | Data used per update | Speed | Stability |
|---|---|---|---|
| Batch GD | All data | Slow | Stable |
| Stochastic GD | One sample | Fast | Noisy |
| Mini-batch GD | Small batch (32–512) | Balanced | Balanced |

### Q9. What is the learning rate and why does it matter?

The learning rate controls the size of each step taken during gradient descent.

- **Too high:** Overshoots the minimum, loss oscillates or diverges
- **Too low:** Converges very slowly, may get stuck in local minima
- **Best practice:** Start at 1e-3, use learning rate schedulers, or use Adam optimiser which adapts the rate automatically

### Q10. What is regularisation? Explain L1 vs L2.

Regularisation adds a penalty to the loss function to discourage overly complex models.

| | L1 (Lasso) | L2 (Ridge) |
|---|---|---|
| Penalty | Sum of absolute weights | Sum of squared weights |
| Effect on weights | Drives some exactly to 0 | Shrinks all weights proportionally |
| Feature selection | ✅ Yes — sparse solution | ❌ No — keeps all features |
| When to use | Many irrelevant features | All features somewhat useful |
| ElasticNet | Combines both L1 and L2 | |

### Q11. What is the difference between classification and regression?

| | Classification | Regression |
|---|---|---|
| Output | Discrete class label | Continuous numeric value |
| Examples | Fraud/not fraud, churn/not churn | House price, sales forecast |
| Loss function | Cross-entropy, log loss | MSE, MAE, RMSE |
| Evaluation | Accuracy, precision, recall, AUC | RMSE, MAE, $R^2$ |

### Q12. What is a loss function? Give examples.

A loss function measures how wrong the model's predictions are. Training minimises this.

| Loss | Used for | Formula (simplified) |
|---|---|---|
| MSE | Regression | Mean of $(\text{actual} - \text{predicted})^2$ |
| MAE | Regression (outlier-robust) | Mean of |
| Binary cross-entropy | Binary classification | $-[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$ |
| Categorical cross-entropy | Multi-class | $-\Sigma\, y \cdot \log(\hat{y})$ |
| Hinge loss | SVM | $\max(0, 1 - y \cdot \hat{y})$ |

### Q13. Explain the difference between generative and discriminative models.

| | Generative | Discriminative |
|---|---|---|
| Models | P(X,Y) — joint distribution | P(Y |
| Examples | Naive Bayes, GANs, VAEs | Logistic Regression, SVM, Neural Networks |

| | Generative | Discriminative |
|---|---|---|
| Can generate new data? | ✅ Yes | ❌ No |
| Generally better at classification? | Less so | ✅ Yes |

### Q14. What is feature scaling and when is it necessary?

| Method | Formula | Use when |
|---|---|---|
| Standardisation (Z-score) | (x - mean) / std | Data approximately normal, SVM, linear models |
| Min-Max Normalisation | (x - min) / (max - min) | Bounded range needed, neural networks |
| Robust Scaling | (x - median) / IQR | Outliers present |

**Required for:** Gradient descent, SVM, KNN, PCA, neural networks.

**Not required:** Tree-based models (Random Forest, XGBoost, Decision Tree).

### Q15. What is data leakage and why is it dangerous?

Data leakage occurs when information from outside the training set contaminates the model, causing artificially high performance that won't hold in production.

**Common forms:**

- Scaling the entire dataset before splitting (should scale only on train, apply to val/test)
- Including target-correlated features that wouldn't exist at prediction time
- Using future data to predict past events
- Filling nulls using statistics from the full dataset before splitting

> **A model with data leakage looks great in evaluation and fails completely in production. Always fit transformers on training data only.**

### Q16. What is class imbalance and how do you handle it?

When one class has far more samples than another (e.g. 99% legitimate transactions, 1% fraud).

**Handling methods:**

- **Resampling:** SMOTE (oversample minority), random undersampling
- **Class weights:** Tell the model to penalise minority class errors more
- **Threshold tuning:** Lower the classification threshold (0.5 → 0.3) to catch more positives
- **Better metrics:** Use precision, recall, F1, AUC-ROC — not accuracy
- **Collect more data** for the minority class

### Q17. What is the difference between bagging and boosting?

| | Bagging | Boosting |
|---|---|---|
| How | Train models in parallel on bootstrapped samples | Train sequentially, each corrects previous errors |
| Goal | Reduce variance | Reduce bias |
| Examples | Random Forest | XGBoost, AdaBoost, LightGBM, CatBoost |
| Overfitting risk | Low | Higher — tune carefully |
| Performance | Good | Often better |

### Q18. What is a confusion matrix?

| | Predicted: YES | Predicted: NO |
|---|---|---|
| **Actual: YES** | TP ✅ | FN ❌ (missed) |
| **Actual: NO** | FP ❌ (false alarm) | TN ✅ |

- **Precision** = TP / (TP + FP) — of all flagged positives, how many were real?
- **Recall** = TP / (TP + FN) — of all actual positives, how many did we catch?
- **F1** = 2 × P × R / (P + R) — harmonic mean, balances both

### Q19. When would you choose precision over recall, and vice versa?

| Scenario | Prioritise | Why |
|---|---|---|
| Cancer screening | Recall | Missing a case is far worse than a false alarm |
| Spam filter | Precision | False positives (blocking legit email) are more costly |
| Fraud detection | Recall | Missing fraud is worse than reviewing more alerts |

| Scenario | Prioritise | Why |
|---|---|---|
| Recommendation | Precision | Showing irrelevant items frustrates users |

### Q20. What is AUC-ROC?

- **ROC curve:** Plots True Positive Rate vs False Positive Rate at every classification threshold
- **AUC (Area Under Curve):** Single number summary — how well the model discriminates between classes
- **AUC = 0.5:** Random guessing
- **AUC = 1.0:** Perfect classification
- **AUC = 0.85+:** Generally considered good

> **Use AUC-ROC when:** Class imbalance exists and you care about ranking/discrimination ability, not a specific threshold.

## 🔵 Supervised Learning — Q21 to Q40

> 💡 Know the algorithm, when to use it, how it works, and its failure modes. The "how it works" explanation should take 60 seconds, not 10 minutes.

### Q21. How does Logistic Regression work?

Logistic Regression models the probability that an input belongs to a class using the sigmoid function:

```
P(y=1|X) = 1 / (1 + e^(-z))     where z = wᵀX + b
```

- Outputs a probability between 0 and 1
- Trained by minimising binary cross-entropy (log loss)
- Decision boundary: predict class 1 if P > 0.5 (tunable)
- **Assumptions:** Linear relationship between features and log-odds, features not highly correlated

### Q22. What is the difference between Logistic Regression and Linear Regression?

| | Linear Regression | Logistic Regression |
|---|---|---|
| Output | Continuous value | Probability (0–1) |
| Activation | None (identity) | Sigmoid |
| Loss | MSE | Binary cross-entropy |
| Use | Regression | Binary classification |
| Decision boundary | N/A | Linear in feature space |

---

### Q23. How does a Decision Tree work?

A tree that splits the data recursively at each node using the feature and threshold that best separates the classes or minimises impurity.

**Splitting criteria:**

- **Gini impurity:** Measures probability of incorrect classification
- **Entropy / Information Gain:** Measures reduction in uncertainty
- **MSE:** For regression trees

**Pros:** Interpretable, no scaling needed, handles mixed types.

**Cons:** Overfits easily (fix with max_depth, min_samples_split, pruning).

---

### Q24. How does Random Forest work and why is it better than a single tree?

Random Forest builds many decision trees on bootstrapped data samples, using a random subset of features at each split, then aggregates predictions (majority vote for classification, average for regression).

**Why better:**

- Reduces variance through averaging (bagging)
- Feature randomness decorrelates trees — makes ensemble more diverse
- Robust to overfitting and outliers
- Built-in feature importance

**Key hyperparameters:** `n_estimators`, `max_depth`, `max_features`, `min_samples_leaf`

---

### Q25. How does XGBoost work?

XGBoost builds trees sequentially. Each new tree corrects the errors (residuals) of all previous trees. It uses gradient descent in function space — fitting each tree to the negative gradient of the loss.

**What makes it fast and accurate:**

- Regularisation (L1/L2) built in
- Parallel tree construction
- Handles missing values natively
- Column subsampling (like Random Forest)
- Pruning by "max_delta_step"

**Key hyperparameters:** `n_estimators`, `learning_rate`, `max_depth`, `subsample`, `colsample_bytree`, `reg_alpha`, `reg_lambda`

---

### Q26. How does Support Vector Machine (SVM) work?

SVM finds the hyperplane that maximises the margin between classes. Support vectors are the data points closest to the decision boundary.

- **Hard margin:** No misclassifications allowed (only for linearly separable data)
- **Soft margin:** Allows some misclassification (C parameter controls this)
- **Kernel trick:** Projects data into higher dimensions to find a linear separator (RBF, polynomial kernels)

**When to use:** High-dimensional data, text classification, small-to-medium datasets.

**When NOT:** Very large datasets (slow), need probability outputs without calibration.

---

### Q27. Explain the K-Nearest Neighbours algorithm.

KNN classifies a new point by finding the K nearest points in the training data and taking the majority class (classification) or average (regression).

- **No training phase** — lazy learner. All computation at prediction time.
- **Distance metric:** Euclidean (default), Manhattan, cosine
- **Scaling required:** Yes — features on different scales distort distances
- **Curse of dimensionality:** KNN degrades in high dimensions
- **Choosing K:** Small K = high variance, Large K = high bias. Use cross-validation.

---

### Q28. What is Naive Bayes and what makes it "naive"?

Naive Bayes is a probabilistic classifier based on Bayes' theorem. It's "naive" because it assumes all features are independent given the class — which is almost never true in reality.

```
P(Y|X) = P(X|Y) × P(Y) / P(X)
```

**Despite the naive assumption, works well for:** Text classification, spam filtering, sentiment analysis.

**Why:** Even if independence is wrong, class ranking still works for many problems.

---

### Q29. How do you handle multicollinearity?

Multicollinearity is when two or more features are highly correlated, making it hard to isolate individual feature effects.

**Detection:** Correlation matrix, Variance Inflation Factor (VIF > 10 = problem).

**Solutions:**

- Remove one of the correlated features
- PCA to create uncorrelated components
- Ridge regression (L2) — handles collinearity better than OLS
- Use tree-based models (unaffected by multicollinearity)

---

### Q30. What is the difference between Ridge, Lasso, and ElasticNet regression?

| | Ridge (L2) | Lasso (L1) | ElasticNet |
|---|---|---|---|
| Penalty | $\lambda \Sigma w^2$ | $\lambda \Sigma$ | w |
| Sets weights to 0? | ❌ No | ✅ Yes | ✅ Some |
| Feature selection | ❌ No | ✅ Yes | ✅ Partial |
| Best when | All features relevant | Many irrelevant features | Mixed case |

---

### Q31. What is precision-recall tradeoff and how do you tune a threshold?

Lowering the classification threshold (e.g. 0.5 → 0.3) increases recall (catches more true positives) but decreases precision (more false positives).

```python
from sklearn.metrics import precision_recall_curve
precisions, recalls, thresholds = precision_recall_curve(y_true, y_proba)
# Plot and choose threshold based on business priority
```

> **Rule:** Choose the threshold based on the cost of each error type, not arbitrarily.

### Q32. How does Gradient Boosting differ from AdaBoost?

| | AdaBoost | Gradient Boosting |
|---|---|---|
| Error correction | Reweights misclassified samples | Fits new tree to residuals (gradient of loss) |
| Base learner | Stumps (depth-1 trees) | Deeper trees |
| Generalisation | Good | Better |
| Speed | Fast | Slower (but XGBoost/LightGBM are fast) |

### Q33. What are the assumptions of Linear Regression?

1. **Linearity** — relationship between X and y is linear
2. **Independence** — observations are independent
3. **Homoscedasticity** — constant variance of residuals
4. **Normality of residuals** — residuals are normally distributed
5. **No multicollinearity** — features not highly correlated

> **In practice:** Tree-based models and neural networks don't have these assumptions — which is why they often outperform linear models on messy real-world data.

### Q34. What is $R^2$ and what are its limitations?

$R^2$ (coefficient of determination) measures how much variance in y is explained by the model. Ranges from 0 to 1 (can be negative for bad models).

```
R² = 1 - SS_residual / SS_total
```

**Limitations:**

- Always increases when you add features — use Adjusted $R^2$ instead
- High $R^2$ doesn't mean causal relationship
- Poor indicator of prediction quality on new data
- Sensitive to outliers

### Q35. When would you use each algorithm?

| Algorithm | Use when |
|---|---|
| Logistic Regression | Baseline, interpretability required, linearly separable |
| Decision Tree | Need full interpretability, mixed data types |
| Random Forest | Tabular data, robust baseline, feature importance |
| XGBoost / LightGBM | Best performance on tabular data, Kaggle-ready |
| SVM | High-dimensional, text, small dataset |
| KNN | Simple baseline, recommendation systems |
| Naive Bayes | Text classification, fast training |
| Neural Network | Images, text, audio, very large datasets |

### Q36. What is stacking (model stacking)?

Stacking is an ensemble method where predictions from multiple base models become features for a meta-model (blender), which learns how to combine them.

```
Base models: Random Forest, XGBoost, SVM
    ↓ predictions on held-out data
Meta-model: Logistic Regression learns weights for each bas
e model
    ↓
Final prediction
```

### Q37. How do you handle categorical variables?

| Method | When to use |
|---|---|
| One-hot encoding | Low cardinality (≤10 categories), linear models |
| Label encoding | Ordinal categories (Low, Medium, High) |

| Method | When to use |
|---|---|
| Target encoding | High cardinality, tree models (careful — leakage risk) |
| Binary encoding | High cardinality — more memory efficient than one-hot |
| Embedding | Neural networks with high cardinality |

### Q38. What is the difference between hard and soft voting classifiers?

- **Hard voting:** Majority class label from all models
- **Soft voting:** Average predicted probabilities — generally better because it leverages confidence levels

### Q39. How do you choose the number of trees in Random Forest?

Use cross-validation. Performance typically improves with more trees up to a point, then plateaus. 100–500 trees is a typical sweet spot. Monitor OOB (out-of-bag) error during training — when it stops decreasing, you have enough trees.

### Q40. What is early stopping?

A regularisation technique where training stops when validation performance stops improving. Prevents overfitting by not running unnecessary epochs.

```
# XGBoost example
model = xgb.XGBClassifier(n_estimators=1000)
model.fit(X_train, y_train,
          eval_set=[(X_val, y_val)],
          early_stopping_rounds=50)  # stop if no improvement for 50 rounds
```

## 🟠 Unsupervised Learning — Q41 to Q55

### Q41. How does K-Means clustering work?

1. Randomly initialise K cluster centroids
2. Assign each point to the nearest centroid
3. Recompute centroids as mean of assigned points
4. Repeat 2–3 until centroids stop moving

**Choosing K:** Elbow method (plot inertia vs K), Silhouette score.

**Limitations:** Assumes spherical clusters, sensitive to initialisation (use K-Means++), struggles with outliers.

---

### Q42. What is the Elbow Method for choosing K?

Plot inertia (sum of squared distances to nearest centroid) vs number of clusters K. The "elbow" — where inertia stops decreasing sharply — suggests the optimal K.

> **Note:** The elbow isn't always obvious. Combine with Silhouette score (higher = better defined clusters).

---

### Q43. What is the Silhouette Score?

Measures how well each point fits its assigned cluster vs neighbouring clusters. Ranges from -1 to +1.

- **+1:** Point is well inside its cluster
- **0:** Point is on the boundary between clusters
- **-1:** Point is in the wrong cluster

---

### Q44. How does DBSCAN work and when is it better than K-Means?

DBSCAN groups points that are densely packed (within distance epsilon of at least min_samples points). Points in sparse regions are labelled as outliers.

**Better than K-Means when:**

- Clusters are irregularly shaped
- You don't know K in advance
- You need to detect outliers
- Clusters have different densities (use HDBSCAN instead)

---

### Q45. What is PCA and how does it work?

PCA finds the directions of maximum variance in the data (principal components) and projects the data onto a lower-dimensional space.

1. Standardise data
2. Compute covariance matrix

3. Find eigenvectors (principal components) and eigenvalues
4. Sort by eigenvalue (explained variance)
5. Project data onto top K components

**Use for:** Dimensionality reduction, visualisation, removing correlated features before linear models.

---

### Q46. What is the difference between PCA and t-SNE?

| | PCA | t-SNE |
|---|---|---|
| Type | Linear | Non-linear |
| Preserves | Global structure, variance | Local structure, clusters |
| Interpretable? | ✅ Yes | ❌ No |
| Use for | Preprocessing, reduction | Visualisation only |
| Reproducible? | ✅ Yes | ❌ No (random seed needed) |
| Scale to new data? | ✅ Yes | ❌ No |

---

### Q47. What is an autoencoder?

A neural network trained to compress data into a lower-dimensional representation (encoding) and then reconstruct it (decoding). The bottleneck layer is the compressed representation.

**Uses:** Dimensionality reduction, anomaly detection (high reconstruction error = anomaly), denoising, generative models.

---

### Q48. What is anomaly detection?

Identifying data points that are significantly different from the majority.

| Method | How | When |
|---|---|---|
| Isolation Forest | Isolates anomalies with fewer splits | General purpose, fast |
| One-Class SVM | Boundary around normal data | Small datasets |
| Autoencoder | High reconstruction error = anomaly | Complex/image data |
| Statistical | Z-score, IQR | Simple, univariate |
| LOF | Local density vs neighbours | Cluster-based anomalies |

---

### Q49. What is hierarchical clustering?

- **Agglomerative (bottom-up):** Start with each point as its own cluster. Merge closest pairs. Build a dendrogram.
- **Divisive (top-down):** Start with one cluster, recursively split.

**Advantage over K-Means:** Doesn't require specifying K upfront. Dendrogram shows cluster structure at every level.

---

### Q50. What is UMAP and how does it compare to t-SNE?

| | t-SNE | UMAP |
|---|---|---|
| Speed | Slow | Much faster |
| Global structure | Poor | Better preserved |
| Scales to new data | ❌ No | ✅ Yes |
| Hyperparameter sensitivity | High | Lower |
| General recommendation | Use UMAP | ✅ Default choice |

---

### Q51. What is the difference between hard and soft clustering?

- **Hard clustering:** Each point belongs to exactly one cluster (K-Means)
- **Soft clustering:** Each point has a probability of belonging to each cluster (Gaussian Mixture Models)

---

### Q52. What is a Gaussian Mixture Model?

GMM assumes data is generated from a mixture of Gaussian distributions. Fitted using the EM (Expectation-Maximisation) algorithm. Each point gets a soft assignment (probability) to each cluster.

**Advantage over K-Means:** Handles elliptical clusters, provides probabilistic assignments.

---

### Q53. How does the EM algorithm work?

1. **E-step (Expectation):** Compute probability that each point belongs to each cluster given current parameters
2. **M-step (Maximisation):** Update cluster parameters (mean, covariance, weights) to maximise likelihood
3. Repeat until convergence

---

### Q54. What is matrix factorisation and where is it used?

Decomposes a matrix into two lower-rank matrices. Key application: recommendation systems.

```
User-Item rating matrix (m×n) ≈ User matrix (m×k) × Item matrix (k×n)
```

Algorithms: SVD, NMF, ALS. Used in Netflix, Spotify, Amazon recommendations.

---

### Q55. What is the difference between feature selection and feature extraction?

| | Feature Selection | Feature Extraction |
|---|---|---|
| How | Selects a subset of original features | Creates new features from original ones |
| Interpretability | ✅ High — original features kept | ❌ Lower — new features not always intuitive |
| Examples | Lasso, RFE, mutual information | PCA, autoencoders, polynomial features |

---

## 🟣 Model Evaluation — Q56 to Q70

### Q56. When should you use accuracy vs F1 score?

- **Accuracy:** Classes are balanced and both errors are equally costly
- **F1:** Class imbalance OR false positives and false negatives have different costs
- **Example:** 99% legitimate transactions, 1% fraud — a model predicting "legit" always gets 99% accuracy but 0 F1 and catches zero fraud

---

### Q57. What is a learning curve and how do you read it?

Learning curves plot training and validation error vs training set size.

| Gap pattern | Diagnosis |
|---|---|
| Both errors high and close | Underfitting — model too simple |
| Training low, validation high, big gap | Overfitting — need more data or regularisation |
| Both low and close | Good fit |

| Gap pattern | Diagnosis |
|---|---|
| Adding data doesn't help | Reached model's capacity limit |

### Q58. What is RMSE and when would you use MAE instead?

- **RMSE:** Root Mean Squared Error. Penalises large errors heavily. Use when large errors are disproportionately bad.
- **MAE:** Mean Absolute Error. All errors treated equally. More robust to outliers.
- **Rule:** If your target has many outliers, MAE is safer. If large errors are especially costly, use RMSE.

### Q59. What is the difference between micro, macro, and weighted averaging for multi-class metrics?

| | Micro | Macro | Weighted |
|---|---|---|---|
| How | Aggregate TP/FP/FN across all classes | Calculate metric per class, then average | Per-class metric weighted by support |
| Treats classes | Equally by sample count | Equally by class | By frequency |
| Use when | Overall performance | Class imbalance, care about all classes equally | Imbalance but importance ≡ frequency |

### Q60. How do you evaluate a clustering model?

| Metric | Requires labels? | What it measures |
|---|---|---|
| Silhouette Score | ❌ No | Cluster cohesion vs separation |
| Davies-Bouldin Index | ❌ No | Average cluster similarity (lower = better) |
| Calinski-Harabasz | ❌ No | Ratio of between/within cluster variance |
| Adjusted Rand Index | ✅ Yes | Agreement with ground truth labels |
| Inertia | ❌ No | Sum of squared distances to centroid |

### Q61. What is k-fold cross-validation and how does it differ from a holdout set?

- **Holdout:** Split once, evaluate once — results depend heavily on which split you got

- **K-fold:** Split K times, evaluate K times, average results — more reliable estimate, uses all data for both training and validation

### Q62. How do you compare two models statistically?

- Run K-fold CV on both
- Compare mean and standard deviation of scores
- Run a paired t-test or Wilcoxon test on fold-level scores
- A difference is meaningful only if it exceeds the variation across folds

### Q63. What is calibration and why does it matter?

A model is well-calibrated if its predicted probabilities match actual frequencies. "80% confidence" should be correct 80% of the time.

**Tools:** Reliability diagram (calibration curve), Brier Score.

**Fix:** Platt scaling (logistic regression on top of outputs), isotonic regression.

**Why it matters:** If you use model probabilities for business decisions (pricing, prioritisation), a miscalibrated model gives wrong expected values.

### Q64. What is NDCG and when is it used?

Normalised Discounted Cumulative Gain — a ranking metric that measures quality of ranked results, where relevant items appearing higher in the list score better.

**Used in:** Search engines, recommendation systems, any ranking problem.

### Q65. What is the difference between offline and online evaluation?

| | Offline | Online |
|---|---|---|
| Method | Historical data, backtest | A/B test in production |
| Speed | Fast | Slow (need traffic) |
| Reliability | May not reflect real behaviour | Ground truth user response |
| Risk | None | Exposes real users to potentially worse experience |

### Q66. How do you detect and handle outliers before training?

| Method | Approach |
|---|---|
| IQR | Flag values below Q1 - 1.5×IQR or above Q3 + 1.5×IQR |
| Z-score | Flag values beyond 3 standard deviations |
| Isolation Forest | Unsupervised anomaly detection |
| Domain rules | Age > 120 is always wrong regardless of statistics |

**What to do:** Investigate first. Remove if data entry error. Cap (winsorise) if real but extreme. Log-transform to reduce skew.

---

**Q67. What is the holdout method and what are its limitations?**

Holdout splits data once into train/test. Limitations:

- High variance — results depend on which split you got
- Wastes data — test set never used for training
- Solution: Cross-validation for more robust estimates

---

**Q68. What is MAPE and what are its problems?**

Mean Absolute Percentage Error = mean of |actual - predicted| / |actual|.

**Problems:**

- Undefined when actual = 0
- Asymmetric — penalises under-predictions more than over-predictions
- Sensitive to small actuals (1 error on 2 actual = 50% error)

**Alternative:** sMAPE, MASE (Mean Absolute Scaled Error)

---

**Q69. How do you choose the right evaluation metric for a business problem?**

1. **Understand the error costs:** What's worse — false positive or false negative? By how much?
2. **Understand the output type:** Probability? Rank? Binary label?
3. **Understand class balance:** Balanced → accuracy OK. Imbalanced → F1, AUC.
4. **Link to business KPI:** A model that optimises log loss but doesn't improve revenue is the wrong model.

---

**Q70. What is the difference between MSE and MAE and when would you use each?**

| | MSE | MAE |
|---|---|---|
| Formula | mean((y - ŷ)²) | mean( |
| Outlier sensitivity | High (squaring amplifies) | Low (linear) |
| Differentiable at 0 | ✅ Yes | ❌ No |
| When to use | Large errors are especially bad | Robust to outliers, errors equally important |

---

## 🔴 Feature Engineering — Q71 to Q82

> 💡 **Feature engineering is the highest-leverage skill in ML.** A mediocre model with great features beats a great model with mediocre features. Every time.

---

### Q71. What is feature engineering and why does it matter?

Feature engineering is creating, transforming, or selecting input variables to improve model performance. It translates domain knowledge into mathematical representations the model can learn from.

**Examples:**

- Date → day of week, is_weekend, days_since_last_purchase
- Price → log(price) to reduce skew
- Lat/Long → distance to nearest store

---

### Q72. How do you handle high-cardinality categorical features?

- **Target encoding:** Replace category with mean of target. Risk: data leakage (use K-fold encoding)
- **Frequency encoding:** Replace category with its frequency in training data
- **Embeddings:** Learn dense representations (for neural networks)
- **Hashing:** Map to a fixed-size space (fast but loses interpretability)
- **Group rare categories** into an "Other" bucket

---

### Q73. What is target encoding and what are the risks?

Target encoding replaces a categorical value with the mean of the target for that category.

**Risk:** Leakage. If you compute the mean using all rows (including the test row), the model sees the target during training.

**Fix:** Use K-fold target encoding — compute encoding on other folds only.

---

### Q74. When would you apply a log transformation to a feature?

- Feature is right-skewed (long tail of large values)
- Feature has multiplicative relationships (e.g. house price – doubling the size doubles the price)
- Target has high positive skew
- To stabilise variance

```
df['log_revenue'] = np.log1p(df['revenue'])  # log1p handles 0s
```

---

### Q75. What is interaction features?

New features created by combining two or more features, capturing non-linear relationships.

```
df['rooms_per_floor'] = df['rooms'] / df['floors']
df['revenue_per_customer'] = df['revenue'] / df['customer_count']
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)  # generates all pairwise products
```

---

### Q76. How do you handle missing values during feature engineering?

- **Don't just fill — create a missing indicator first:** `df['age_missing'] = df['age'].isna().astype(int)`
- **Numeric:** Fill with median (more robust than mean), mean, or model-based imputation (KNN Imputer)
- **Categorical:** Fill with mode or "Unknown" category
- **Time series:** Forward fill, backward fill, or interpolate

- **Never:** Fill test set using test set statistics — fit on train, transform both

### Q77. What is feature importance and how do you calculate it?

| Method | Model | Interpretation |
|---|---|---|
| Gini importance | Tree models | Average impurity reduction across splits |
| Permutation importance | Any | Performance drop when feature values shuffled |
| SHAP values | Any | Contribution of each feature to each prediction |
| Coefficient magnitude | Linear models | Weight of each feature (after scaling) |

> **SHAP is the gold standard.** Gini importance can be misleading for high-cardinality features.

### Q78. What is the difference between filter, wrapper, and embedded feature selection methods?

| | Filter | Wrapper | Embedded |
|---|---|---|---|
| How | Statistical test on features | Evaluate feature subsets with a model | Feature selection during model training |
| Examples | Correlation, chi-squared, mutual information | RFE, forward selection | Lasso, decision tree |
| Speed | Fast | Slow | Medium |
| Model-aware | ❌ No | ✅ Yes | ✅ Yes |

### Q79. What is principal component analysis used for in feature engineering?

- Create uncorrelated features from correlated ones before linear models
- Reduce dimensionality to avoid curse of dimensionality
- Remove noise (low-variance components)
- Visualise high-dimensional data in 2D/3D

**Not useful for:** Tree-based models (handle correlated features fine), when interpretability is required (components aren't original features)

### Q80. How do you create time-based features from a timestamp?

```
df['hour']        = df['ts'].dt.hour
df['day_of_week']= df['ts'].dt.dayofweek   # 0=Mon, 6=Sun
df['is_weekend'] = df['day_of_week'].isin([5,6]).astype(in
t)
df['month']       = df['ts'].dt.month
df['quarter']     = df['ts'].dt.quarter
df['days_since'] = (df['ts'].max() - df['ts']).dt.days
df['lag_7d']      = df['value'].shift(7)    # 7-day lag
df['roll_7d_avg']= df['value'].rolling(7).mean()
```

**Q81. What is oversampling and undersampling?**

- **Undersampling:** Randomly remove majority class samples. Fast but loses data.
- **Oversampling:** Randomly duplicate minority class samples. Easy but creates exact copies.
- **SMOTE (Synthetic Minority Oversampling Technique):** Creates synthetic minority samples by interpolating between existing ones. Generally better than random oversampling.

```
from imblearn.over_sampling import SMOTE
X_res, y_res = SMOTE().fit_resample(X_train, y_train)
```

**Q82. What is the difference between one-hot encoding and ordinal encoding?**

- **One-hot encoding:** Each category becomes a binary column. No order assumed. ✅ For nominal categories (colour, country, product type)
- **Ordinal encoding:** Categories mapped to integers (Low=0, Medium=1, High=2). Order is encoded. ✅ For ordered categories.
- **Warning:** Never apply ordinal encoding to nominal categories — it implies a false ordering that misleads linear models.

## ⚫ Deep Learning & NLP — Q83 to Q95

**Q83. How does backpropagation work?**

Backpropagation computes gradients of the loss with respect to each weight using the chain rule, flowing backwards from the output layer to the input layer. These gradients are then used by gradient descent to update weights.

```
Forward pass: input → prediction → loss
Backward pass: loss → ∂loss/∂weights (chain rule) → weight
update
```

### Q84. What is dropout and how does it prevent overfitting?

During training, randomly set a fraction p of neurons to zero at each forward pass. This forces the network to learn redundant representations — no single neuron can be relied upon.

- At test time, all neurons are active and weights are scaled by (1-p)
- Typical rates: p = 0.2–0.5
- Acts like training an ensemble of exponentially many sub-networks

### Q85. What are activation functions and why are they needed?

Activation functions introduce non-linearity, enabling networks to learn complex patterns.

| Function | Formula | Use |
|---|---|---|
| ReLU | max(0, x) | Hidden layers (default) |
| Leaky ReLU | max(0.01x, x) | Dying ReLU problem |
| Sigmoid | 1/(1+e^-x) | Binary output layer |
| Softmax | e^x / Σe^x | Multi-class output layer |
| Tanh | (e^x - e^-x)/(e^x + e^-x) | RNNs, between -1 and 1 |

### Q86. What is batch normalisation?

Normalises layer inputs to have zero mean and unit variance, then scales with learned parameters gamma and beta. Applied between layers during training.

**Benefits:** Faster training, allows higher learning rates, slight regularisation effect, reduces sensitivity to initialisation.

### Q87. What is a convolutional neural network (CNN) and when do you use it?

CNNs use convolutional layers to detect local patterns (edges, textures, shapes) by sliding a learned filter across the input. Pooling layers downsample. Fully connected layers classify.

**Use for:** Images, video, spatial data, anything where local patterns matter.

---

### Q88. What is a recurrent neural network (RNN) and what problem does LSTM solve?

- **RNN:** Processes sequences with a hidden state carrying information across time steps. Suffers from vanishing gradient problem — can't remember long-range dependencies.
- **LSTM:** Adds gates (forget, input, output) to control what information to keep or discard across many time steps. Solves the vanishing gradient problem.

---

### Q89. What is the Transformer architecture?

Transformers process entire sequences in parallel using self-attention, which allows every token to attend to every other token. Key components:

- **Self-attention:** Each token computes relevance to all other tokens
- **Multi-head attention:** Multiple attention heads capture different relationships
- **Positional encoding:** Injects sequence order (transformers have no inherent order)
- **Feed-forward layers:** Transform each token's representation independently

---

### Q90. What is the attention mechanism?

Attention allows the model to weight the importance of different parts of the input when producing each output element.

```
Attention(Q, K, V) = softmax(QKᵀ / √d_k) V
```

- Q (Query): What we're looking for
- K (Key): What each token represents
- V (Value): What we return for each token

---

### Q91. What is transfer learning?

Take a model trained on a large dataset (e.g. ImageNet, web text), then fine-tune on a smaller task-specific dataset. The pre-trained model already knows general representations — fine-tuning adapts the last layers to the new task.

**Why it works:** Low-level features (edges, textures for images; grammar, semantics for text) are useful across many tasks.

### Q92. What is the difference between BERT and GPT?

| | BERT | GPT |
|---|---|---|
| Architecture | Encoder only | Decoder only |
| Attention direction | Bidirectional | Left-to-right (causal) |
| Pre-training task | Masked Language Modelling | Next token prediction |
| Best for | Understanding (classification, NER, QA) | Generation (text, code) |

### Q93. What are word embeddings and why are they better than one-hot encoding?

Word embeddings represent words as dense vectors in a continuous space where semantically similar words are close together.

- One-hot: 50,000-dimensional sparse, no relationships captured
- Word2Vec/GloVe: 300-dimensional dense, "king - man + woman ≈ queen"
- Contextual (BERT): Different embedding for same word in different contexts

### Q94. What is the vanishing gradient problem?

In deep networks, gradients become exponentially small as they propagate backwards through many layers. Early layers receive near-zero gradients and barely update — making deep networks hard to train.

**Solutions:** ReLU activations, batch normalisation, residual connections (ResNet), LSTM gates, gradient clipping.

### Q95. What is a GAN?

Generative Adversarial Network — two networks trained together:

- **Generator:** Tries to generate realistic fake data
- **Discriminator:** Tries to distinguish real from fake

They compete until the generator produces data indistinguishable from real. Used for image synthesis, data augmentation, style transfer.

## 🟡 ML System Design — Q96 to Q110

> 💡 **This is the senior-level differentiator.** Anyone can train a model. Very few can design a system that stays reliable, fair, and improving in production.

**Q96. How would you approach building an ML model from scratch?**

1. **Understand the business problem** — what decision does the model enable? What's the success metric?
2. **Define the ML task** — classification, regression, ranking?
3. **Understand the data** — EDA, quality check, availability at prediction time
4. **Establish a baseline** — rule-based, simple heuristic, or average
5. **Feature engineering** — domain knowledge into model inputs
6. **Model selection and training** — start simple, iterate
7. **Evaluation** — offline metrics aligned to business goals
8. **Deploy** — API, batch, or embedded
9. **Monitor** — data drift, model drift, business KPI

**Q97. What is model drift and how do you detect it?**

- **Data drift:** Distribution of input features changes over time
- **Concept drift:** Relationship between features and target changes
- **Label drift:** Distribution of target variable changes

**Detection:** Monitor feature statistics (KS test, PSI), compare prediction distribution, track business KPIs, set alerts on metric drops.

**Q98. How would you design a recommendation system?**

| Approach | How | When |
|---|---|---|
| Collaborative filtering | Find similar users/items | Dense interaction data |
| Content-based | Features of items + user preferences | Cold start, rich metadata |
| Two-tower model | Separate user and item encoders | Large-scale, real-time |
| Matrix factorisation | Decompose user-item matrix | General purpose |
| Hybrid | Combine multiple signals | Production systems |

**Key problems to address:** Cold start (new users/items), scalability, feedback loops.

---

**Q99. What is A/B testing in the context of ML?**

- Split users randomly into control (Model A) and treatment (Model B)
- Run for sufficient time to reach statistical significance
- Measure business metric, not just ML metric
- Check for novelty effects (early boost from something new, not better)
- Guard against multiple comparisons problem

---

**Q100. How do you handle the cold start problem?**

- **New user cold start:** Popularity-based recommendations, onboarding questions, demographic features
- **New item cold start:** Content-based features, item metadata, human editorial curation
- **Hybrid:** Combine content signals immediately, blend in collaborative filtering as data accumulates

---

**Q101. How would you make an ML model explainable?**

| Method | Works for | Output |
|---|---|---|
| SHAP | Any model | Per-feature contribution per prediction |
| LIME | Any model | Local linear approximation |
| Feature importance | Tree models | Global feature ranking |
| Attention weights | Transformers | Which tokens the model attended to |
| Partial dependence plots | Any | Marginal effect of one feature |
| Simpler model | — | Use interpretable model (logistic regression, decision tree) |

---

**Q102. What is MLOps?**

MLOps is the practice of deploying, monitoring, and maintaining ML models in production reliably.

**Key components:**

- **Data versioning** (DVC)
- **Experiment tracking** (MLflow, W&B)
- **Model registry** (version control for model artifacts)
- **CI/CD for ML** (automated retrain → test → deploy pipeline)
- **Monitoring** (data drift, model performance, latency)
- **Feature store** (centralised, versioned feature computation)

---

### Q103. How do you serve a model in production?

| Method | Use when | Latency |
|---|---|---|
| REST API | Real-time, interactive | Low ms |
| Batch scoring | Offline, periodic | Hours/days |
| Streaming | Near-real-time on event streams | Low seconds |
| Edge inference | On-device, no internet | Sub-ms |

---

### Q104. What is feature store and why is it important?

A feature store is a centralised system for storing, computing, and serving ML features. It ensures:

- Same features used in training and serving (training-serving skew prevention)
- Features computed once and reused across multiple models
- Backfilling historical features
- Point-in-time correct feature retrieval

---

### Q105. How do you detect and mitigate bias in ML models?

**Detection:**

- Slice performance by demographic groups
- Confusion matrix per subgroup
- Disparate impact analysis

**Mitigation:**

- Pre-processing: Balance training data, remove proxy features
- In-training: Fairness constraints, adversarial debiasing

- Post-processing: Equalize odds by adjusting thresholds per group

### Q106. What is online learning vs batch learning?

| | Batch Learning | Online Learning |
|---|---|---|
| Updates | On full dataset | On each new sample or mini-batch |
| When to use | Stable data, can retrain periodically | Rapidly changing data, real-time adaptation |
| Examples | Most offline training | Ad click prediction, stock trading |
| Risk | Staleness | Sensitivity to bad data |

### Q107. What is the difference between model accuracy and business impact?

A model with 95% accuracy that doesn't change user behaviour has zero business impact. A model with 80% accuracy that increases conversion 15% is a success.

**Bridge the gap:**

- Define the decision the model enables
- Estimate impact per decision quality unit
- A/B test to measure actual behaviour change
- Track business KPI, not just ML metric

### Q108. How do you handle very large datasets that don't fit in memory?

- **Chunked processing:** Read and process in batches
- **Dask or Spark:** Distributed computation
- **Feature hashing:** Map features to fixed-size arrays without materialising all categories
- **Sampling:** Use a representative sample for initial modelling
- **Cloud ML:** Vertex AI, SageMaker — managed large-scale training
- **Incremental learning:** `partial_fit()` in sklearn for online algorithms

### Q109. What is shadow deployment?

New model runs in parallel with production model but its outputs are not served to users. Real traffic is used to compare predictions silently before committing to the new model. Eliminates risk of exposing users to untested model.

### Q110. How do you decide when to retrain a model?

- **Scheduled retraining:** Every N days regardless (simple, predictable)
- **Performance-triggered:** Retrain when validation metric drops below threshold
- **Drift-triggered:** Retrain when data drift exceeds a threshold (PSI, KS test)
- **Event-triggered:** Major business change, new data source, product change

> **Best practice:** Automate the trigger, validate before promoting the new model, keep rollback ready.

## ✅ You're Interview-Ready When...

| Category | Can you do this without notes? |
|---|---|
| **Fundamentals** | Explain bias-variance tradeoff, cross-validation, gradient descent, regularisation |
| **Algorithms** | Explain how Random Forest, XGBoost, and Logistic Regression work in 60 seconds each |
| **Evaluation** | Name the right metric for any given business problem and justify it |
| **Features** | Explain why feature engineering matters more than algorithm choice |
| **Deep Learning** | Explain backprop, dropout, attention, and transfer learning clearly |
| **System Design** | Walk through an end-to-end ML system including drift monitoring |
| **Business** | Translate a business problem into an ML task with the right success metric |